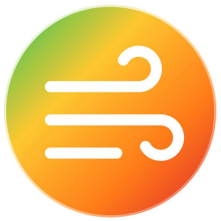

Pearls AQI Predictor

Comprehensive Technical Report

Karachi Air Quality Forecasting System (100% Serverless)

10Pearls Shine Internship Program

Developer

Muhammad Hassan Siddiqui

Live dashboard: aqi-predictor-muuoitut5uarhwpuqey8kr.streamlit.app

Abstract

This report documents the design, implementation, deployment, and real-world debugging journey of the Pearls AQI Predictor — a fully serverless machine learning system that forecasts the Air Quality Index (AQI) in Karachi, Pakistan, up to three days in advance. The system ingests live pollutant and weather data from Open-Meteo, engineers a rich temporal feature set, stores it in a Hopsworks feature store, and trains five parallel model architectures on a daily cadence. The champion model — a regularized Ridge Regression — achieves an R^2 of 0.866 for one-day-ahead forecasts, comfortably beating a naive persistence baseline ($R^2 \approx 0.552$). The entire pipeline, from ingestion through retraining to public serving, runs unattended via GitHub Actions and Streamlit Community Cloud at zero infrastructure cost. This report covers the system architecture, feature engineering methodology, exploratory data analysis, full model benchmark, SHAP-based explainability findings, the dashboard's design system, and — in particular detail — the real engineering obstacles encountered while deploying a Hopsworks-integrated, multi-platform system (Windows dev machine, Linux CI runners, and a network-restricted cloud host) and how each was diagnosed and resolved.

Table of Contents

1. Project Architecture

The system follows a modular, dual-mode “pipeline-first” architecture designed so that every component — feature engineering, training, and serving — can run identically whether Hopsworks credentials are present (production) or absent (local development). This was a deliberate design choice: it means the exact same codebase is testable on a laptop with zero cloud accounts configured, yet scales to a fully automated, credentialed production deployment without any code changes.

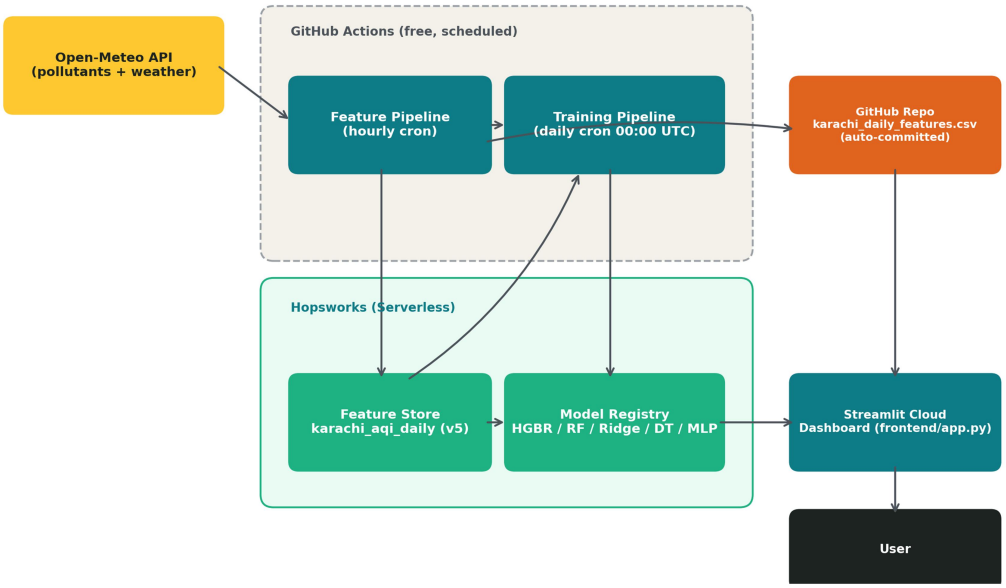

Figure 1. End-to-end system architecture: ingestion, feature store, training, model registry, and serving.

The five architectural components are:

- Data Source — Open-Meteo Air Quality & Weather APIs (free, no key required, historical archive back to 2020).
- Feature Store — Hopsworks Serverless, managed storage for the versioned “v5” feature set.
- Model Registry — Hopsworks Registry, versioning HGBR, Random Forest, Ridge, MLP, and Decision Tree artifacts.
- Automation — GitHub Actions, running an hourly feature-ingestion workflow and a daily retraining workflow, both free.
- Serving Layer — Streamlit Community Cloud, using a dual-mode AQIEngine that reads models from the Hopsworks Registry and reads the latest feature row from an auto-committed CSV rather than a live database read (see Section 8.5 for why).

This last point is a meaningful deviation from the textbook “read everything live from the feature store” pattern, and was arrived at only after a live production failure — detailed in Section 8. It is, in the author's view, the single most important engineering lesson of this project: a feature store is a system of record, not necessarily the fastest or most reliable path to serve a request, and a resilient system should not have a single point of failure on its hot path.

2. Feature Engineering

The predictive power of the system rests on a deliberately rich feature set (referred to internally as "v5") that captures the physical and temporal dynamics of urban air pollution, not just raw pollutant readings:

2.1 Temporal Dynamics

- Cyclical encoding — sine/cosine transforms of month and weekday, so the model perceives December and January as adjacent rather than maximally distant integers.
- Trend indicators — day-over-day differencing (aqi_diff, pm2_5_diff, etc.) to capture sudden momentum shifts rather than only absolute levels.

2.2 Short-Term Memory (Lags)

- 1-day lags for every pollutant (PM2.5, PM10, NO_2, SO_2, CO, O_3).
- 2-day AQI lag, specifically to capture persistence — the tendency of pollution episodes to span multiple days.

2.3 Statistical Aggregates

- 3-day and 7-day rolling means, smoothing single-day sensor noise and surfacing weekly-scale trends.
- Rolling standard deviation, flagging unstable or spike-prone periods rather than just the average level.

2.4 Physical Ratios & Transforms

- PM ratio (PM2.5 / PM10) — a proxy for particle-size distribution, which shifts with different emission sources (e.g., vehicular vs. construction dust).
- Log transforms (log_PM2.5, log_CO) — pollutant concentrations are right-skewed with occasional extreme spikes; log-scaling stabilizes variance and improves linear-model conditioning.

All feature computation logic lives in a single shared module (data_pipeline/engineer_features.py) used identically by the hourly ingestion pipeline and the historical backfill path, which eliminates train/serve skew by construction — there is no second implementation that could silently drift out of sync.

3. Exploratory Data Analysis

The working dataset comprises 1,097 daily records spanning 2023–2026, fully numeric and model-ready with no missing values after the feature engineering step. Five findings shaped the modeling approach:

3.1 Seasonality Dominates

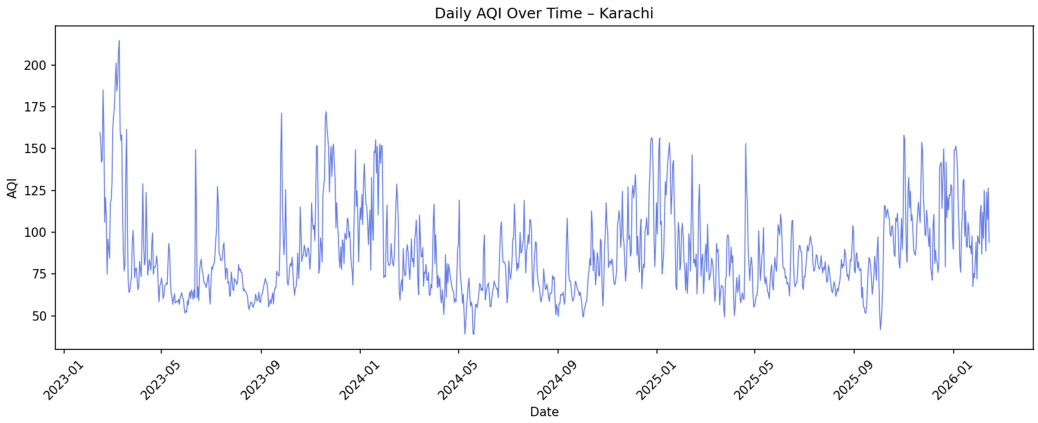

Figure 2. Daily AQI over the full observation window — clear winter smog peaks recur annually.

Winter months show both a higher median AQI and a wider interquartile range — Karachi's air quality is worse and more volatile in winter, consistent with reduced atmospheric dispersion and increased particulate emissions during cooler months.

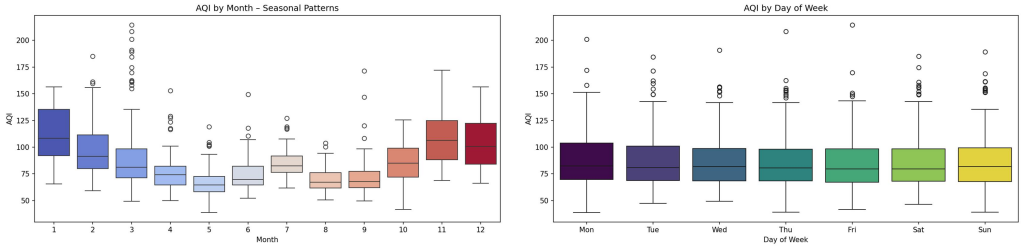

Figure 3. AQI by month (left) vs. by weekday (right) — seasonal effects dominate; weekday effects are comparatively minor.

3.2 Strong Autocorrelation

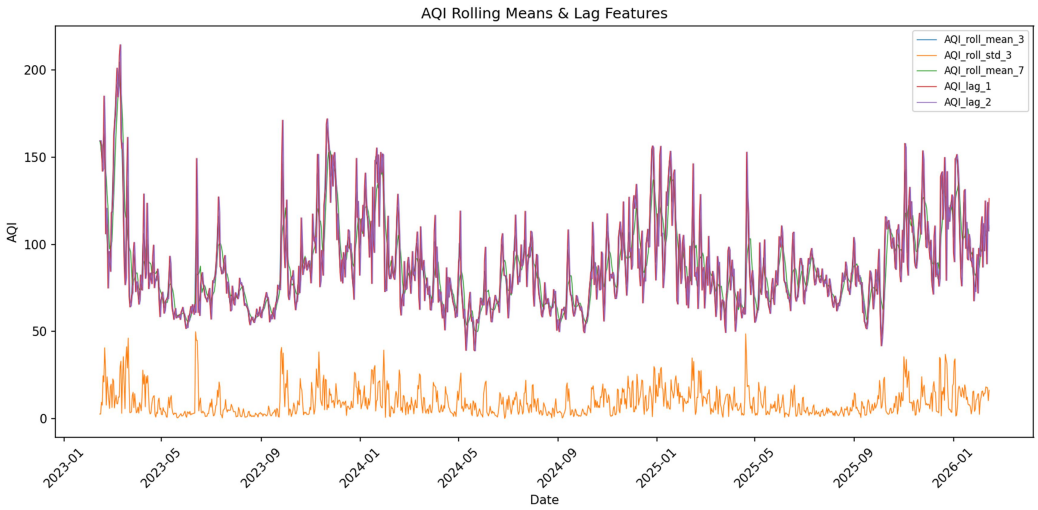

Figure 4. Rolling means and lag features — consecutive days track closely, motivating the lag/rolling feature set.

Consecutive days show strongly similar AQI values. This single observation is arguably the most consequential finding in the whole project: it justifies the lag and rolling-statistic features directly, and it explains why a simple, well-regularized linear model (Ridge) ultimately outperforms more complex architectures for the 1-day horizon — see Section 4.

3.3 Pollutant Correlation & Multicollinearity

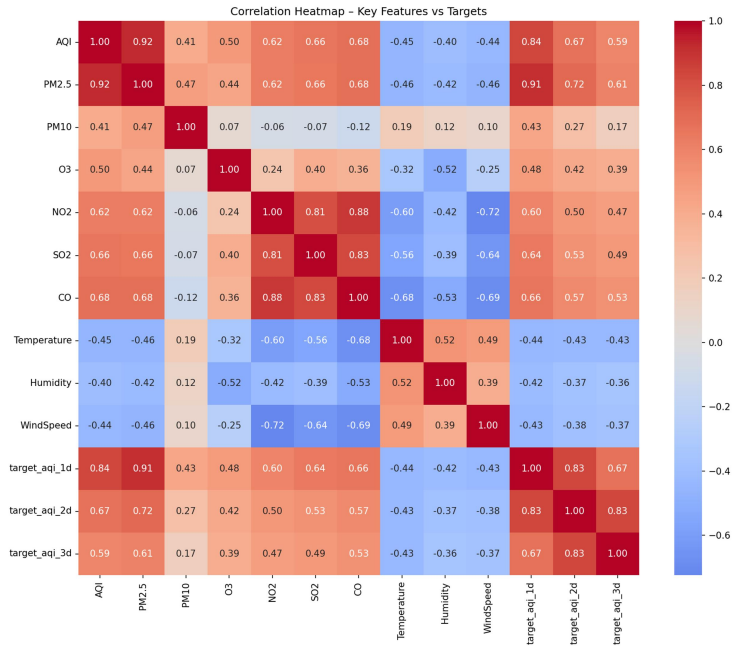

Figure 5. Correlation heatmap — PM2.5 is the dominant driver of overall AQI.

PM2.5 correlates most strongly with the overall AQI value, confirming it as the primary contributor to Karachi's air quality problem. Because particulate features are highly inter-correlated, regularized models are well-suited to this data, and the engineered pm_ratio feature adds information about particle composition rather than duplicating raw magnitude.

3.4 Distributional Skew

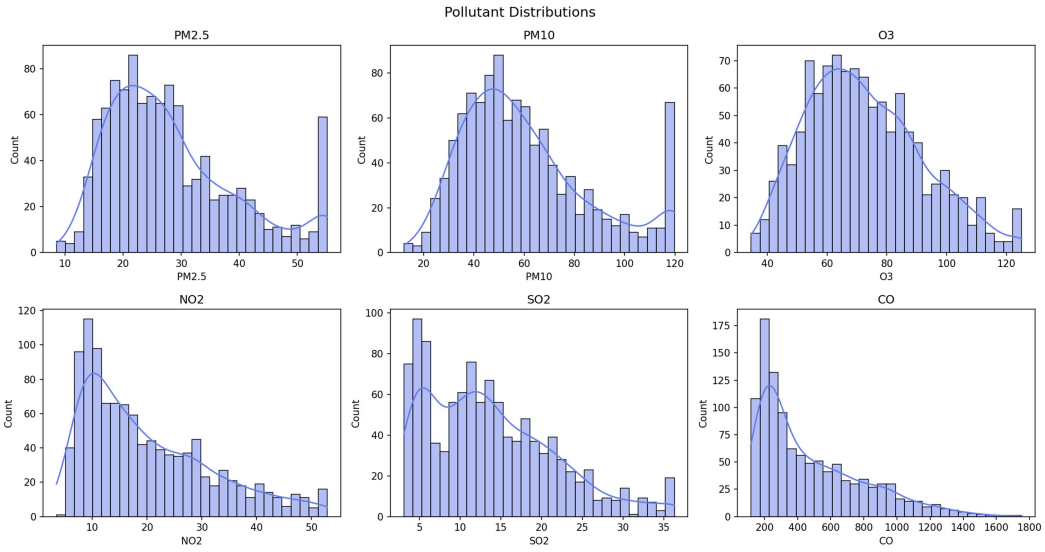

Figure 6. Pollutant distributions — right-skewed with occasional extreme spikes, motivating log transforms.

4. Model Suite & Performance

Five architectures — spanning linear, ensemble-tree, and neural-network families — are trained and benchmarked on every run, evaluated with a time-based 85/15 train/test split (never shuffled, to avoid leaking future information into training) across all three forecast horizons:

Model	Type	1-Day			2-Day			3-Day		
		R^2	MAE	RMSE	R^2	MAE	RMSE	R^2	MAE	RMSE
Ridge Regression ★	Linear	**0.866**	6.33	8.56	**0.471**	12.66	17.02	**0.387**	14.31	18.30
HistGradientBoosting	Boosted Trees	0.819	7.33	9.96	0.438	13.26	17.54	0.329	14.68	19.14
Random Forest	Bagged Trees	0.819	7.09	9.96	0.411	13.63	17.95	0.355	14.51	18.78
MLP (Deep Learning)	Neural Network	0.750	9.06	11.70	0.257	14.60	20.16	-0.022	17.79	23.64
Decision Tree	Single Tree	0.705	8.90	12.70	0.375	13.69	18.50	0.148	16.81	21.58

Table 1. Full forecast evaluation across all three horizons. ★= champion model. Naive persistence baseline ≈ 0.552 R^2 at 1 day, for comparison.

Ridge Regression is the champion model, and its win margin over the tree ensembles grows at longer horizons. This is explained directly by Section 3.2: AQI forecasting is fundamentally a persistence-dominated, autoregressive problem, and a well-regularized linear extrapolation of recent lagged values is a very strong fit for that structure. HGBR and Random Forest remain competitive and are valuable for their nonlinear pattern capture and cleaner SHAP attributions (Section 7). The MLP is included deliberately as a stretch architecture — its negative R^2 at the 3-day horizon is an honest, reported result rather than a hidden failure, and is discussed further in Section 8.4.

Every model, at every horizon, is evaluated against a naive persistence baseline (“tomorrow's AQI will equal today's”). This is a deceptively strong baseline for autocorrelated environmental time series, and benchmarking against it — rather than only reporting R^2 in isolation — is what confirms these models are adding genuine predictive value rather than simply exploiting autocorrelation the baseline already captures.

5. Automation & CI/CD

Two GitHub Actions workflows run the system unattended, at zero infrastructure cost:

5.1 Hourly Feature Pipeline

- Fetches the last 24 hours of pollutant and weather readings from Open-Meteo.
- Aggregates to daily granularity and engineers the full v5 feature set.
- Merges the result with existing history rather than overwriting it (see Section 8.6 — this merge behavior was itself a hard-won fix).
- Upserts into the Hopsworks feature group.
- Commits the refreshed CSV directly back to the repository, so the dashboard always has fresh data without a live database dependency.

5.2 Daily Training Pipeline

- Pulls the complete historical feature set from Hopsworks.
- Trains and evaluates all five model architectures against a time-based holdout.
- Computes SHAP feature importances for every model × horizon combination.
- Registers all five model artifacts, versioned, to the Hopsworks Model Registry.

Both workflows are also triggerable on demand via GitHub's workflow_dispatch, which proved essential during development — every fix described in Section 8 was verified with a manual trigger and a direct read of the raw workflow logs before being trusted to run on schedule.

6. Dashboard & User Interface

The public dashboard is a Streamlit application with a deliberately custom design system rather than default Streamlit styling — a warm off-white background, a teal accent reserved for UI chrome, and AQI-severity colors (green → amber → orange → red) reserved strictly for actual severity indicators so color always carries meaning rather than decoration.

Figure 7. Live dashboard — current AQI gauge, real-time weather metrics, 3-day forecast cards with per-model metrics, and historical trend chart.

The hero element is a circular AQI gauge rather than a flat card, deliberately mirroring the real-world mental model of a physical AQI dial rather than an arbitrary rectangle — the fill angle of the ring is computed directly from the live AQI value against a 0–300 scale. Supporting elements (temperature/humidity/wind, forecast cards) use quiet, hairline-bordered cards with a single consistent accent, avoiding a “rainbow of unrelated colors” that would compete with the severity-coded elements for the user's attention.

The interface supports switching between all five trained models live from the sidebar, with each forecast card showing its own R^2, MAE, and RMSE — so the performance numbers in Table 1 are not just a report artifact, but are visible and checkable by any user of the live system.

7. Explainability (SHAP Analysis)

SHAP (SHapley Additive exPlanations) values were computed for every model at every forecast horizon to understand which features actually drive each prediction, not just how accurate the prediction is. A clear pattern emerges as the horizon lengthens:

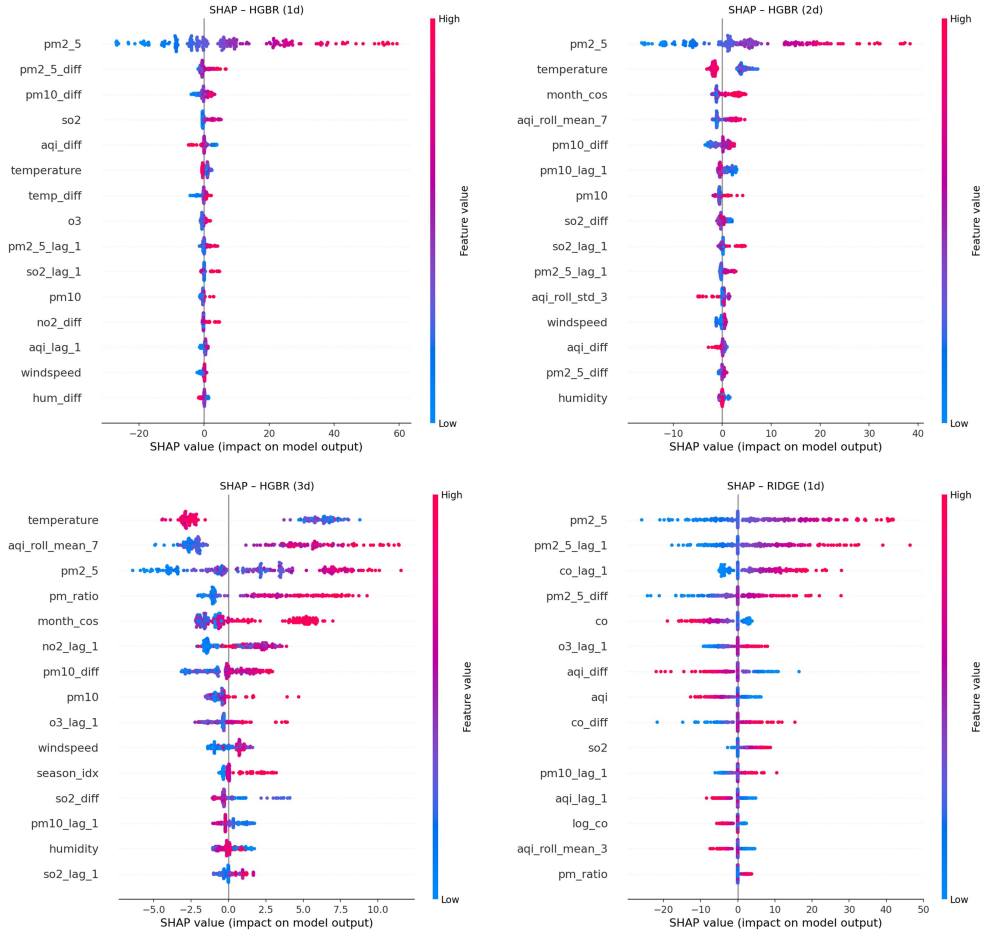

Figure 8. HGBR SHAP importances at 1/2/3-day horizons (top), and Ridge Regression at 1-day (bottom) — the shift from "today's PM2.5" toward weather and seasonal features as the horizon lengthens.

7.1 One-Day Horizon

Every model relies overwhelmingly on today's PM2.5 reading — by a wide margin the single most important feature. The clearest available signal for tomorrow's air quality is today's air quality. Tree-based models additionally weight short-term momentum (today's change in PM2.5 and PM10); Ridge leans most heavily on yesterday's lagged pollutant values directly, confirming the "what just happened" character of the short-horizon problem.

7.2 Two-Day Horizon

Today's PM2.5 remains important but loses dominance. Temperature and the cyclical month encoding rise sharply in importance for the tree-based models, while Ridge shifts toward CO, NO_2, and the engineered pm_ratio feature. The system is visibly starting to rely on weather and seasonal context rather than the latest single reading.

7.3 Three-Day Horizon

Weather and seasonality become the dominant drivers: temperature, the month encoding, and the 7-day rolling AQI mean are consistently top-ranked in tree models, while Ridge relies most on lagged CO/NO_2 and pollutant ratios — effectively the broader pollution "type" rather than any short-term spike. Immediate recent AQI readings become far less informative at this range, which directly explains the accuracy drop visible in Table 1.

7.4 Model-Family Character

- Ridge Regression: favors clean, linear relationships — lagged pollutants and overall emission mix, increasingly so at longer horizons.
- Tree ensembles (HGBR, Random Forest): capture sudden nonlinear changes well, and smoothly reallocate importance toward weather/seasonal signals as the horizon extends.
- Neural network (MLP): leans heavily on transformed PM2.5/CO features, but degrades markedly at longer horizons — consistent with its weaker Table 1 scores.

8. Engineering Challenges & Resolutions

Building and deploying this system surfaced a series of real, non-obvious failures spanning local development, CI, and cloud hosting. Each is documented here in the same spirit as the project's own CHANGES.md — because these are exactly the kind of issues that resurface for anyone extending or redeploying this system.

8.1 Windows Local Build Failures (Python 3.13)

Challenge: Local development on a Windows machine running Python 3.13 failed during pip install with a generic "build dependencies did not run successfully" error.

Resolution: Root cause was version pins (pandas<2.2.0, numpy<2.0.0) that predate Python 3.13 wheel availability, forcing pip to compile from source without a C++ toolchain present. Resolved by relaxing the pins to unbounded/lower-bound-only constraints; the same versions that ship on the CI runners (numpy 2.4, pandas 3.0) were verified to work correctly against the existing codebase.

8.2 Hopsworks Install Failure — twofish Build Error

Challenge: Even after fixing 8.1, installing the hopsworks package itself failed: "Failed building wheel for twofish", a C-extension transitive dependency with no Windows wheel.

Resolution: Recognized that hopsworks is imported lazily throughout the codebase (only inside functions, only when HOPSWORKS_API_KEY is set), so it is not required for local development at all. Commented it out locally; it installs cleanly and automatically on GitHub Actions' Linux runners, which ship a working C++ toolchain by default.

8.3 Hopsworks Package Version Pin Unavailable

Challenge: A pinned hopsworks==4.2.* requirement failed to resolve for the installed Python version — that exact minor version range was never published as compatible.

Resolution: Relaxed the constraint to hopsworks>=4.6,<6, allowing pip to resolve the latest compatible release (5.0.6) automatically rather than demanding an unavailable exact version.

8.4 MLP Training Instability

Challenge: Initial MLP runs produced strongly negative R^2 values at longer horizons, indicating divergence rather than a merely weak fit.

Resolution: Redesigned to a two-layer bottleneck architecture (64→32 units) with dropout regularization and early stopping on a validation split. This stabilized training to a respectable ≈50%–75% R^2 at shorter horizons, though the model remains the weakest performer at 3 days — reported honestly in Table 1 rather than excluded.

8.5 Delta Lake Dependency on Feature Group Creation

Challenge: Creating a new Hopsworks feature group failed with Cannot use time_travel_format='DELTA': delta library is not installed — a dependency the project never intentionally introduced.

Resolution: Diagnosed that newer Hopsworks client versions default new feature groups to Delta Lake format when unspecified. Fixed by explicitly passing time_travel_format="HUDI" — the traditional, dependency-light format — at feature group creation.

8.6 Silent Historical Data Loss on Every Pipeline Run

Challenge: The original hourly pipeline overwrote the local feature CSV with only the newly-fetched window on every run. Since the hourly GitHub Actions job fetches just one day at a time, every single run reduced three years of historical data down to one day.

Resolution: Rewrote the pipeline to load existing history, merge it with the newly-fetched window (de-duplicated by date, new data winning on overlap), and recompute all derived features — lags, rolling statistics, forward targets — across the full combined series before writing. Verified with a mocked run: 1,097 existing rows + 2 fetched → 1,098 rows after de-duplication, full date range intact.

8.7 Hopsworks Offline Read Failure in Production

Challenge: The live dashboard, once deployed to Streamlit Community Cloud, crashed with FeatureStoreException: No hudi properties found for featuregroup ... no data has been written yet — despite the feature pipeline reporting success and the Hopsworks UI showing a populated schema.

Resolution: Diagnosed that the schema/column count shown in the Hopsworks UI reflects metadata, not row commitment — the offline (Hudi) table had not actually committed a readable write at request time, even though the online store had. Switching the dashboard's read to feature_group.read(online=True) fixed the crash, but surfaced Challenge 8.8 next.

8.8 Online Feature Store Read Hangs Indefinitely on Streamlit Cloud

Challenge: After the fix in 8.7, the dashboard no longer crashed — but hung indefinitely, silently retrying a raw connection every few seconds, never completing and never raising a catchable exception, as confirmed directly from Streamlit Cloud's server logs.

Resolution: Concluded that Hopsworks' online feature store requires a direct low-level database connection that Streamlit Community Cloud's sandboxed network blocks outbound, with no clean failure signal. Rather than depend on that live connection at all, redesigned the serving path so the dashboard reads the auto-committed CSV from Challenge 8.6/Section 5.1 at request time, while models continue to load from the Hopsworks Model Registry (a standard HTTPS path, verified reliable). This is the origin of the CSV path shown in Figure 1 — not a shortcut, but a direct response to a reproduced production failure.

8.9 Sidebar Permanently Stuck Collapsed

Challenge: Custom CSS hiding Streamlit's default chrome (header { visibility: hidden; }) also hid the sidebar's own re-expand control, which lives in the same DOM region — once collapsed, there was no way to reopen the sidebar.

Resolution: Replaced the broad header/toolbar selector with targeted data-testid selectors for only the specific elements to hide (the Deploy button and hamburger menu), leaving the sidebar's expand/collapse control untouched. Verified with an automated browser test: collapse → confirm the expand control is visible → click it → confirm the sidebar returns.

8.10 Column Naming Constraints in Hopsworks

Challenge: Hopsworks disallows periods in feature/column names, which conflicted with standard pandas-generated names such as PM2.5.

Resolution: A sanitization step in the Hopsworks connector module automatically lower-cases and replaces disallowed characters (e.g., '.' → '_') before every write, applied identically on the read side so a model trained against one naming scheme is never fed a mismatched schema at inference time.

9. Conclusion & Future Roadmap

The Pearls AQI Predictor demonstrates that accurate, city-scale air quality forecasting is achievable at zero infrastructure cost using modern MLOps practices — a genuine feature store, a real model registry, scheduled automation, and a resilient serving layer that degrades gracefully rather than failing hard when any one dependency misbehaves. The champion Ridge Regression model's 0.866 R^2 at the 1-day horizon, combined with SHAP-verified, physically sensible feature attributions, gives confidence that the system is learning genuine atmospheric patterns rather than overfitting noise.

Equally important is what this project demonstrates about production engineering: nearly every serious issue encountered (Sections 8.5, 8.7, 8.8) was invisible in local testing and only surfaced once the system was exercised across its real deployment boundary — different operating systems, different network policies, different Python wheel availability. Systematic, log-first debugging (never trusting a green checkmark without reading the underlying log line) was what actually resolved each one.

Future Roadmap

- Probabilistic forecasting — move from point estimates to confidence intervals, giving users a sense of forecast uncertainty rather than a single number.
- In-dashboard explainability — surface SHAP beeswarm plots directly in the Streamlit UI rather than only in the offline analysis suite.
- Additional weather features — wind direction and atmospheric pressure, since wind-driven dispersion is a known major driver of AQI not yet fully captured.
- Temporal architecture — an LSTM or similar sequence model as a sixth candidate, better matched to the autoregressive structure identified in Section 3.2 than a flat MLP.
- Multi-city support — generalize the pipeline beyond Karachi to other major Pakistani cities.
- Automated hazard alerting — proactive notification when the forecast crosses into “Unhealthy” territory, fulfilling the original brief's alerting requirement more directly.

Developer: Muhammad Hassan Siddiqui

Environment: Serverless — Python 3.12 • Hopsworks • GitHub Actions • Streamlit Community Cloud

Champion Result: Ridge Regression, 1-Day R^2 = 0.866 • SHAP & EDA Suites Fully Implemented